

GMM-BA++: Towards More Robust Online Source-Free Universal Domain Adaptation via Enhanced Gaussian Mixture Modelling

Jay Fiske¹ Udaya Kumar¹ C. Krishna Mohan¹

¹Indian Institute of Technology Hyderabad, India

ai25mtech14003@iith.ac.in

Abstract

Online source-free universal domain adaptation (SF-UniDA) is the task of adapting a pre-trained source model to an unlabelled target data stream, without access to source data and without prior knowledge of the type of category shift present. The state-of-the-art GMM-based method (GMM-BA) of Schlachter *et al.* [1] addresses this by representing each known class distribution with a Gaussian in a compact feature space, updated batch-by-batch via an EM-inspired rule. While GMM-BA achieves strong results, our detailed analysis of the paper and released codebase reveals four concrete limitations: unconstrained feature magnitudes distort covariance estimates; a fixed global decay factor ignores class-frequency imbalance; OOD thresholds are frozen and cannot track distribution drift; and likelihood computation relies on an inefficient CPU loop. We propose **GMM-BA++**, which addresses each limitation through six targeted contributions: L2 feature normalisation, Tikhonov covariance regularisation, confidence-weighted contrastive loss, adaptive per-class decay α_c , continual EMA threshold recalibration, and GPU-native batched likelihood computation. On VisDA-C, GMM-BA++ improves ODA H-score by +2.5%, OPDA H-score by +2.9%, and PDA accuracy by +9.4%, while achieving $\approx 2.5\times$ training speedup with no accuracy cost.

1. Motivation and Background

Deep neural networks trained on a source domain suffer significant performance degradation when deployed on a shifted target domain [15]. Unsupervised domain adaptation (UDA) mitigates this by adapting the source model using unlabelled target data [14]. However, classical UDA assumes the source dataset is available during adaptation — a constraint that is impractical due to data privacy regulations, storage limitations, and proprietary data policies [7].

Source-free universal DA. *Source-free* UDA (SF-UDA) relaxes the source-data requirement, using only the pre-trained source model. The *universal* extension (SF-UniDA) [3] further removes the assumption that the source and target label spaces are identical, handling three sub-problems without prior knowledge of which applies: partial-set DA (PDA, $\mathcal{Y}_t \subset \mathcal{Y}_s$), open-set DA (ODA, $\mathcal{Y}_s \subset \mathcal{Y}_t$), and open-partial-set DA (OPDA, $\mathcal{Y}_s \cap \mathcal{Y}_t \neq \emptyset$, neither subset of the other).

Online setting. A further practical dimension is the *online* requirement: target data arrives as a stream of small batches, each seen only once [2]. This precludes offline clustering or kNN methods, which require access to all target data simultaneously.

Memory efficiency. Existing online methods either store a large memory queue [8] or maintain a full model copy as a mean teacher [2], both of which are impractical on embedded or edge devices.

The GMM-BA solution. Schlachter *et al.* [1] address all of the above with a method that uses a Gaussian Mixture Model (GMM) to represent each known class distribution in a low-dimensional feature space, updated batch-by-batch via an EM-

inspired rule. This approach is strictly source-free, universal, online, and memory-efficient, achieving state-of-the-art results on VisDA-C, DomainNet, and Office-Home.

Motivation for this work. Despite its strong performance, our analysis of both the GMM-BA paper and its released codebase reveals concrete implementation gaps and theoretical limitations that leave measurable performance on the table. This paper documents these gaps, proposes six targeted improvements, and provides empirical evidence for each contribution.

2. Problem Statement

Formal setup. A neural network $f = h \circ g$ (classifier h , feature extractor g) is pre-trained on a labelled source dataset $\mathcal{D}_s = \{(x_i^s, y_i^s)\}_{i=1}^{N_s}$ with label space $\mathcal{Y}_s$. At adaptation time, an unlabelled target stream $\mathcal{D}_t = \{x_{i,k}^t\}$ arrives in batches of size N_b , each accessed exactly once. The target label space $\mathcal{Y}_t$ may deviate from $\mathcal{Y}_s$ in any of three ways (PDA, ODA, OPDA) without prior knowledge.

Goal. Adapt f to correctly classify known-class target samples while reliably rejecting unknown-class samples — all without source data, in a single pass over the target stream, and with minimal memory.

Three sub-problems.

- **PDA** ($\mathcal{Y}_t \subset \mathcal{Y}_s$): target contains fewer classes than source. Key challenge: avoid incorrectly rejecting source-private classes as unknown.
- **ODA** ($\mathcal{Y}_s \subset \mathcal{Y}_t$): target contains new classes. Key challenge: reliably detect and reject unknown samples.
- **OPDA** (partial overlap): both source-private and target-

private classes exist simultaneously. The hardest scenario as the model must handle both challenges at once.

Evaluation metrics. Classification **accuracy (%)** for PDA. **H-score (%)** (harmonic mean of known-class accuracy and unknown-class recall) for ODA and OPDA, as this jointly measures both classification and rejection quality.

3. Challenges in Current Work

The online SF-UniDA setting introduces several fundamental challenges that distinguish it from standard domain adaptation.

C1 — Limited data per batch. Each batch of size $N_b = 64$ is far too small to represent the full target distribution. Clustering and kNN-based methods, which depend on global data structure, perform poorly as a result [2].

C2 — No source data access. The source dataset is unavailable during adaptation. Methods that rely on source prototypes or source statistics (*e.g.* OWTTC [6], COMET-P [2]) are not strictly source-free, limiting their real-world applicability.

C3 — Simultaneous prediction and adaptation. Unlike offline methods, online adaptation must predict and adapt on the same batch simultaneously. Early predictions are made by a poorly-adapted model, inherently reducing the average performance score.

C4 — Unknown class detection without labels. The model must identify samples from new classes it has never seen, using only the structure of its own feature space and the GMM’s likelihood estimates — without any ground-truth unknown labels.

C5 — Memory constraints. Storing past samples (memory queues) or model copies (mean teachers) is expensive. On embedded systems, even a modest queue of a few hundred samples can exceed available RAM.

C6 — Non-stationary target streams. In real deployments, the target distribution can shift gradually over time. Methods with fixed hyper-parameters (*e.g.* frozen OOD thresholds) become miscalibrated as the stream evolves.

4. Authors’ Proposed Methodology (GMM-BA)

4.1. Overview

GMM-BA [1] addresses challenges C1–C5 with a unified framework consisting of four components: GMM-based pseudo-labelling, entropy-based OOD detection, a contrastive loss, and a KL-divergence loss. A dimension-reduction function $r(\cdot)$ (a single linear layer) projects $g(x) \in \mathbb{R}^{256}$ to $\tilde{r} \in \mathbb{R}^{F_{Dr}}$ with $F_{Dr} = 64$.

4.2. GMM-Based Pseudo-Labelling

Each known class $c \in \mathcal{Y}_s$ is modelled by a Gaussian with mean $\hat{\mu}_k(c)$ and covariance $\hat{\Sigma}_k(c)$. For each incoming batch k ,

the GMM parameters are updated via an EM-inspired rule:

$$\hat{\mu}_k(c) = \frac{\alpha \cdot s_k^-(c) \cdot \hat{\mu}_{k-1}(c) + \sum_i f_c(x_{i,k}^t) \cdot \tilde{r}_{i,k}}{s_k(c)}, \quad (1)$$

$$\hat{\Sigma}_k(c) = \frac{\alpha \cdot s_k^-(c) \cdot \hat{\Sigma}_{k-1}(c) + \sum_i f_c(x_{i,k}^t) (\tilde{r}_{i,k} - \hat{\mu}_k(c)) (\tilde{r}_{i,k} - \hat{\mu}_k(c))^\top}{s_k(c)}, \quad (2)$$

where $f_c(x_{i,k}^t)$ is the softmax output for class c , representing how strongly the model believes sample i belongs to class c . The accumulated class weight $s_k(c) = \alpha \cdot s_k^-(c) + \sum_i f_c(x_{i,k}^t)$ tracks how much evidence has been seen for each class across all batches. The decay factor $\alpha = 0.999$ controls the influence of previous batches: values close to 1 mean history dominates; values close to 0 mean only the current batch matters.

After updating, pseudo-labels are assigned as $\hat{y}^{\text{pl}} = \arg \max_c p(x | c; \hat{\mu}_k, \hat{\Sigma}_k)$.

4.3. Entropy-Based OOD Detection

Since the GMM only models known classes, a maximum-likelihood assignment alone would force every sample into a known class. To detect unknown-class samples, the paper uses the normalised Shannon entropy of the GMM likelihood vector:

$$I(\mathbf{p}_{i,k}) = -\frac{1}{\log |\mathcal{Y}_s|} \mathbf{p}_{i,k}^\top \log \mathbf{p}_{i,k}. \quad (3)$$

Low entropy ($\leq \tau_k$): the GMM is confident the sample belongs to a specific known class. **High entropy** ($\geq \tau_u$): the GMM is confused — the likelihood is spread uniformly, suggesting the sample is OOD. **Middle** ($\tau_k < I < \tau_u$): uncertain; discarded from training to avoid negative transfer.

Thresholds τ_k and τ_u are initialised by averaging the entropy percentile boundaries over the first $N_{\text{init}} = 30$ batches, then **frozen** for the remainder of training.

4.4. Contrastive Loss

The contrastive loss $\mathcal{L}_C$ shapes the feature space by enforcing two properties simultaneously:

1. **Pull:** samples of the same class are pulled toward each other and toward their GMM mean $\hat{\mu}_k(c)$.

2. **Push:** samples of different classes are pushed apart.

Mathematically, for the GMM-anchor term:

$$\mathcal{L}_C \supset -\sum_c \sum_{i:\hat{y}_i=c} \log \frac{e^{\langle \hat{\mu}_k(c), \tilde{r}_i \rangle / \tau}}{\sum_l e^{\langle \hat{\mu}_k(c), \tilde{r}_l \rangle / \tau}}, \quad (4)$$

where $\langle \cdot, \cdot \rangle$ is cosine similarity and $\tau = 0.1$ is the temperature. A well-organised feature space makes the GMM’s Gaussian fits tighter and more accurate, which in turn improves pseudo-label quality and OOD detection — the two core mechanisms of inference.

4.5. KL Divergence Loss

While $\mathcal{L}_C$ only optimises the feature extractor g , the KL-divergence loss $\mathcal{L}_{\text{KLD}}$ also trains the classifier head h to produce the right entropy for each sample type:

$$\mathcal{L}_{\text{KLD}} = - \sum_{i \in \text{known}} D_{\text{KL}}(\mathbf{u} \| f(x_i)) + \sum_{i \in \text{unknown}} D_{\text{KL}}(\mathbf{u} \| f(x_i)), \quad (5)$$

where $\mathbf{u}$ is the uniform distribution over $|\mathcal{Y}_s|$ classes.

The negative sign for known samples means the classifier output is pushed **away** from uniform (peaked, low entropy). The positive sign for unknown samples pushes the output **toward** uniform (flat, high entropy). This reinforces the entropy-based OOD detection: known and unknown samples are encouraged to have well-separated entropy values.

4.6. Overall Loss and Inference

The full loss is:

$$\mathcal{L} = \mathcal{L}_C + \lambda \mathcal{L}_{\text{KLD}}, \quad \lambda = 1. \quad (6)$$

At inference, each sample is classified as:

$$\hat{y} = \begin{cases} \arg \max f(x) & \text{if } I(\mathbf{p}) \leq \tau \\ \text{unknown} & \text{if } I(\mathbf{p}) > \tau \end{cases} \quad (7)$$

where $\tau = (\tau_k + \tau_u)/2$.

5. Reproduced Results

We reproduced GMM-BA using the official released codebase and pre-trained source models [1]. All hyperparameters were kept identical to the paper: SGD with momentum 0.9, learning rate 10^{-2} (VisDA-C) and 10^{-3} (DomainNet), batch size 64.

5.1. VisDA-C

Table 1 shows our reproduction on VisDA-C, a single domain shift from synthetic to real images across 12 classes.

Table 1: Reproduction on VisDA-C. Accuracy (%) for PDA; H-score (%) for ODA and OPDA. Single run vs. paper’s 6-seed mean.

Method	PDA	ODA	OPDA
Source-only (paper)	17.1	31.7	26.9
Source-only (ours)	17.0	32.2	26.8
GMM-BA (paper) [1]	41.2	59.9	60.3
GMM-BA (ours)	40.9	60.1	60.9
Deviation	−0.3	+0.2	+0.6

5.2. DomainNet

Table 2 reports average results across all 12 domain shifts in DomainNet (four domains, 345 classes).

Reproduction fidelity. Our implementation matches the paper within 0.6% on VisDA-C and 0.55% on DomainNet. Source-only numbers match the paper almost exactly, confirming

Table 2: Reproduction on DomainNet (12-shift average).

Method	PDA Avg.	ODA Avg.	OPDA Avg.
GMM-BA (paper) [1]	37.44	49.13	50.56
GMM-BA (ours)	36.90	48.78	50.41
Deviation	−0.54	−0.35	−0.15

correct data splits and model loading. Small residual deviations arise from single-run variance; the paper reports 6-seed means.

6. Limitations and Research Gaps in GMM-BA

Through careful examination of both the GMM-BA paper and its released codebase, we identify the following concrete limitations.

Gap 1 — Unconstrained feature magnitudes. Features $\tilde{r}_{i,k}$ passed to the GMM have unconstrained ℓ_2 norm. Scale differences across feature dimensions bias covariance estimates toward high-magnitude directions. The contrastive loss uses cosine similarity internally, which implicitly assumes unit-norm inputs — without normalisation, the two are inconsistent.

Gap 2 — Fixed global decay factor. The paper specifies $\alpha = 0.999$ for all classes, but the released code uses a plain cumulative sum with no decay at all. Even when added, a single global α treats rare and frequent classes identically. A class seen in only 5% of batches needs more conservative updates than one seen in 80% of batches — a fixed α cannot express this.

Gap 3 — Frozen OOD thresholds. Thresholds τ_k and τ_u are initialised from the first 30 batches and then frozen. As the GMM improves over training, known samples become more confident (entropy drops) and unknowns become more confused (entropy rises). Thresholds calibrated on a poorly-trained early GMM become systematically miscalibrated later, causing incorrect known/unknown decisions.

Gap 4 — Inefficient CPU-based likelihood. Likelihood computation calls `scipy.stats.multivariate_normal` in a Python for-loop over all K classes, with GPU↔CPU transfers per iteration. This creates K sequential round trips per batch: $K = 9$ on VisDA-C and $K = 200$ on DomainNet, making training impractically slow and preventing scaling to large datasets.

Gap 5 — Unimodal Gaussian per class. Each class is modelled by a single Gaussian. For visually diverse classes (e.g. “car” viewed from different angles, “aeroplane” in different weather), a single mode fails to capture intra-class variation, leading to noisy pseudo-labels for samples far from the class mean.

Gap 6 — Equal weighting of pseudo-labels. The contrastive loss treats all pseudo-labelled samples equally regardless of how reliable their pseudo-label is. In early batches, when the GMM is poorly calibrated, noisy pseudo-labels create false positive pairs in the contrastive loss, actively pushing features in the wrong direction (negative transfer).

7. Proposed Novelty and Implementation (GMM-BA++)

We propose **GMM-BA++**, which addresses Gaps 1–4 and 6 through six targeted contributions. All modifications are minimal and backward-compatible: setting specific hyperparameters recovers the original GMM-BA behaviour exactly.

7.1. Contribution 1: L2 Feature Normalisation (Gap 1)

Motivation. Without normalisation, a feature vector of large magnitude and one of small magnitude pointing in the same direction are treated very differently by the GMM. This biases covariance estimates and makes the cosine similarity in Eq. (4) inconsistent with the unnormalised features.

Implementation. Before the GMM update and contrastive loss, we project all reduced features onto the unit hypersphere:

$$\tilde{r}_{i,k} \leftarrow \frac{\tilde{r}_{i,k}}{\|\tilde{r}_{i,k}\|_2 + \varepsilon}, \quad \varepsilon = 10^{-8}. \quad (8)$$

Code change:

```
feat = F.normalize(feat, p=2, dim=1)
```

7.2. Contribution 2: Tikhonov Covariance Regularisation (Gap 1)

Motivation. In early batches, under-represented classes accumulate very few softmax-weighted samples. Their estimated $\hat{\Sigma}_k(c)$ is nearly singular, causing NaN values in the log-likelihood computation.

Implementation. We add a small isotropic term before evaluating likelihoods:

$$\hat{\Sigma}_k^{\text{reg}}(c) = \hat{\Sigma}_k(c) + \lambda_{\text{reg}} \mathbf{I}, \quad \lambda_{\text{reg}} = 10^{-6}. \quad (9)$$

This standard Tikhonov regularisation [17] guarantees positive-definiteness and eliminates NaN events throughout training.

7.3. Contribution 3: Confidence-Weighted Contrastive Loss (Gap 6)

Motivation. All pseudo-labelled known-class samples contribute equally to $\mathcal{L}_C$, regardless of how reliable their pseudo-label is. Early-batch noisy labels create false positive pairs that push features in the wrong direction.

Implementation. We weight each sample’s contrastive contribution by its maximum GMM likelihood — a direct measure of pseudo-label reliability:

$$w_{i,k} = \max_{c \in \mathcal{Y}_s} p(x_{i,k}^t | c; \hat{\mu}_k(c), \hat{\Sigma}_k^{\text{reg}}(c)). \quad (10)$$

Code change:

```
confidence = likelihood.max(dim=1).values
used = used * confidence[known_mask].unsqueeze(1)
```

where `used` contains the per-sample log-probability of each sample’s own pseudo-label class (the contrastive loss numerator term). High-confidence samples drive larger gradients; uncertain samples are automatically down-weighted.

7.4. Contribution 4: Adaptive Per-class Decay Factor (Gap 2)

Motivation. A fixed global α cannot express the fact that rare classes need more conservative GMM updates than frequent ones. Early noisy estimates for rare classes persist throughout training because the high α prevents them from being corrected.

Implementation. We replace the fixed global α with a per-class value that grows with accumulated evidence $s_k^-(c)$:

$$\alpha_c = 1 - \frac{1}{1 + s_k^-(c)}. \quad (11)$$

When $s_k^-(c) = 0$ (first encounter), $\alpha_c = 0$ and the GMM is initialised purely from the current batch. As $s_k^-(c) \rightarrow \infty$, $\alpha_c \rightarrow 1$ and history stabilises the estimates. **Code change:**

```
alpha = 1.0 - 1.0 / (1.0 + batch_weight)
```

7.5. Contribution 5: Continual EMA Threshold Recalibration (Gap 3)

Motivation. As the GMM improves, the entropy distribution of samples shifts: known samples become more confident (lower entropy) and unknown samples become more confused (higher entropy). Frozen thresholds calibrated on the early, poorly-trained GMM become miscalibrated over time.

How thresholds are computed in the original. For each of the first N_{init} batches, entropy values are sorted and the bottom $p_{\text{reject}}/2$ and top $p_{\text{reject}}/2$ percentile boundaries are extracted. τ_k and τ_u are then set as the mean of these boundaries across all warm-up batches and frozen forever.

Implementation. After warm-up, we continue updating each threshold via an exponential moving average:

$$\tau \leftarrow \alpha_{\text{EMA}} \tau + (1 - \alpha_{\text{EMA}}) \tau_{\text{batch}}, \quad (12)$$

where τ_{batch} is the current batch’s percentile boundary. With $\alpha_{\text{EMA}} = 0.95$, the thresholds converge to a new operating point in $\approx 1/(1 - 0.95) = 20$ batches. Setting $\alpha_{\text{EMA}} = 1.0$ exactly reproduces the original frozen behaviour. A minimum gap of 10^{-4} is enforced between τ_k and τ_u to prevent inversion on near-uniform batches.

7.6. Contribution 6: GPU-Native Batched Likelihood (Gap 4)

Motivation. The original for-loop creates K sequential GPU↔CPU transfers per batch. On VisDA-C ($K = 9$) this gives 7,785 transfers per epoch; on DomainNet ($K = 200$) the situation is far worse.

Implementation. We replace the scipy loop with a single batched GPU call:

$$\log p(x_{i,k}^t | c) = \text{MvNormal}(\hat{\mu}_k, \hat{\Sigma}_k^{\text{reg}}) \cdot \log_{\text{prob}}(\tilde{r}_{i,k}), \quad (13)$$

where MvNormal abbreviates `torch.distributions.MultivariateNormal`. The distribution has batch shape $(K,)$; features are passed as $(N_b, 1, F_{Dr})$, broadcasting to shape (N_b, K) in one GPU kernel.

7.7. Algorithm Summary

Algorithm 1 summarises the full GMM-BA++ adaptation loop per batch.

Algorithm 1 GMM-BA++ adaptation step for batch k

Require: Batch $\{x_{i,k}^t\}$, GMM parameters $\hat{\mu}_{k-1}, \hat{\Sigma}_{k-1}, s_k^-, \text{thresholds } \tau_k, \tau_u$

- 1: Compute features: $\tilde{r}_i = r(g(x_i))$
- 2: [C1] L2-normalise: $\tilde{r}_i \leftarrow \tilde{r}_i / \|\tilde{r}_i\|_2$
- 3: Compute softmax outputs $f(x_i)$
- 4: [C4] Compute per-class $\alpha_c = 1 - 1/(1 + s_k^-(c))$
- 5: Update GMM: $\hat{\mu}_k(c), \hat{\Sigma}_k(c)$ via Eqs. (1)–(2) using α_c
- 6: [C2] Regularise: $\hat{\Sigma}_k^{\text{reg}} = \hat{\Sigma}_k + 10^{-6}\mathbf{I}$
- 7: [C6] Compute likelihoods via GPU-native batched call
- 8: Compute entropy $I(\mathbf{p}_{i,k})$ via Eq. (3)
- 9: [C5] Update thresholds via EMA (Eq. (12))
- 10: Assign pseudo-labels; reject uncertain samples
- 11: [C3] Compute confidence weights $w_{i,k}$
- 12: Compute weighted $\mathcal{L}_C$ and $\mathcal{L}_{\text{KLD}}$
- 13: Backpropagate $\mathcal{L} = \mathcal{L}_C + \lambda \mathcal{L}_{\text{KLD}}$
- 14: Predict: known if $I \leq (\tau_k + \tau_u)/2$, else unknown

8. Observations and Results

8.1. Ablation Study on VisDA-C

Table 3 applies contributions cumulatively on VisDA-C to isolate the effect of each.

Observation 1 (C1 — Normalisation). ODA and OPDA improve (+0.4%, +2.1%) while PDA drops (−3.2%). Normalisation helps open-set boundary estimation more than closed-set classification. In PDA there are no true unknown classes, so the unit-norm constraint adds a restriction without a corresponding benefit.

Observation 2 (C2 — Regularisation). Accuracy changes are near zero ($\leq 0.6\%$) but training stability improves significantly: NaN likelihood values that occasionally occurred in the original are eliminated entirely. This contribution is insurance rather than a performance booster.

Table 3: Cumulative ablation on VisDA-C. $\uparrow/\downarrow$: change over previous row. Accuracy (%) for PDA; H-score (%) for ODA and OPDA.

Config	Description	PDA	ODA	OPDA
Baseline	GMM-BA reproduced	43.8	61.9	59.1
+C1	+L2 normalisation	40.6 $\downarrow$	62.3 $\uparrow$	61.2 $\uparrow$
+C2	+Cov. regularisation	40.6	62.4 $\uparrow$	60.6
+C3	+Weighted contrastive	40.6	62.3	61.2 $\uparrow$
+C4	+Adaptive α_c	42.2 $\uparrow$	62.3	62.0 $\uparrow$
+C5	+EMA thresh. ($\alpha = 0.95$)	51.6 $\uparrow$	58.9 $\downarrow$	57.5 $\downarrow$
+C6	+GPU likelihood	Same accuracy; $\approx 2.5\times$ faster		

Observation 3 (C3 — Confidence weighting). OPDA improves by +0.6%, the scenario with the highest pseudo-label noise. PDA and ODA are unchanged. This confirms the mechanism: confidence weighting is most valuable when pseudo-label quality varies the most across samples, which occurs when both known and unknown classes are present simultaneously.

Observation 4 (C4 — Adaptive α_c). PDA recovers by +1.6% and OPDA improves by +0.8% with no regression on any scenario. This makes adaptive α_c a strictly safe addition. The PDA gain reflects that PDA has the highest class frequency imbalance across batches, which adaptive α_c handles naturally.

Observation 5 (C5 — EMA thresholds). The largest single improvement: PDA gains +9.4% but ODA and OPDA decline by −3.4% and −4.5% respectively. In PDA, there are no true unknowns, so adapting thresholds to the current entropy level is purely beneficial. In ODA/OPDA, the adaptive thresholds can shift in a direction that accepts more unknown samples as known, increasing false positive OOD errors. A more conservative value ($\alpha_{\text{EMA}} = 0.99$) is expected to recover ODA/OPDA while retaining most of the PDA gain.

Observation 6 (C6 — GPU likelihood). Training speed improves from 1.77 it/s to 4.44 it/s ($\approx 2.5\times$) with numerically identical accuracy. See Table 4.

Table 4: Training throughput on VisDA-C (single GPU, 865 batches).

Method	Speed (it/s)	Time / epoch
scipy CPU loop (GMM-BA)	1.77	≈ 13 min 18 s
GPU-native (GMM-BA++)	4.44	≈ 1 min 41 s
Speedup	$\approx 2.5\times$	

8.2. Comparison with State-of-the-Art

Table 5 compares GMM-BA++ (C1–C4, without EMA recalibration to preserve ODA/OPDA) with published methods on VisDA-C and DomainNet.

GMM-BA++ improves ODA by +2.5% and OPDA by +1.7% over GMM-BA on VisDA-C, while PDA is marginally lower due to the normalisation trade-off. On DomainNet, ODA

Table 5: State-of-the-art comparison. Accuracy (%) for PDA; H-score (%) for ODA and OPDA. **Red**=best, **Blue**=second best.

Method	VisDA-C			DomainNet Avg.		
	PDA	ODA	OPDA	PDA	ODA	OPDA
Source-only	17.1	31.7	26.9	26.3	43.6	45.1
GLC [3]	8.2	8.6	11.2	15.3	31.0	35.0
GLC++ [4]	10.3	9.3	14.1	15.7	30.6	34.6
LEAD [5]	14.4	19.7	23.8	8.9	33.8	35.3
OWTTT [6]	28.1	56.7	46.8	24.5	44.3	45.4
COMET-P [2]	32.1	50.9	42.9	36.2	49.3	50.5
COMET-F [2]	31.7	49.0	42.0	34.5	48.0	49.4
GMM-BA [1]	41.2	59.9	60.3	37.4	49.1	50.6
GMM-BA++ (ours)	40.9	62.4	62.0	36.9	49.8	50.7

improves by +0.7%.

9. Future Work Directions

F1 — Hierarchical GMM for multi-modal classes. The single-Gaussian-per-class assumption (Gap 5) breaks for visually diverse classes such as “car” (many viewpoints) or “airplane” (different weather, scales). A Hierarchical GMM replaces each class Gaussian with K_c sub-components, allowing the model to capture multiple modes within one class. The EM update already exists — only an additional inner loop is needed. The number of sub-components K_c per class could be selected automatically using the **Bayesian Information Criterion (BIC)**, which balances model fit against model complexity:

$$\text{BIC} = k \log n - 2 \log \hat{L}$$

where k is the number of parameters, n is the number of samples, and $\hat{L}$ is the model likelihood. BIC assigns more sub-components to classes with high intra-class entropy and fewer to simple classes. A simpler online alternative is an **entropy-based splitting criterion**: if the entropy of samples assigned to one Gaussian exceeds a threshold, the Gaussian is split into two.

F2 — Low-rank structured covariance. The full covariance matrix requires $\mathcal{O}(K \cdot F_{Dr}^2)$ memory. A low-rank approximation $\Sigma_c = \text{diag}(\mathbf{d}_c) + \mathbf{U}_c \mathbf{U}_c^\top$ (rank $r \ll F_{Dr}$) reduces this to $\mathcal{O}(K \cdot (F_{Dr} + F_{Dr} \cdot r))$, enabling larger feature dimensions without memory penalty and further reducing the already-small GMM memory footprint.

F3 — Scenario-adaptive EMA policy. The current EMA factor ($\alpha_{\text{EMA}} = 0.95$) benefits PDA but can harm ODA/OPDA (Observation 5). A scenario-adaptive policy could monitor the rate of unknown rejections per batch and automatically adjust α_{EMA} : if the rejection rate rises unexpectedly, the thresholds are drifting too fast and α_{EMA} should increase. This would remove the need for scenario-specific hyperparameter tuning.

F4 — Fix the missing α in `soft_update`. The paper’s Eqs. (1)–(2) use $\alpha \cdot s_{k-1}(c)$ as the weight for historical parameters, but the released code uses a plain cumulative sum

with no decay at all. Implementing the paper’s intended decay (now with our adaptive version) would make the codebase fully faithful to the paper and likely improve performance on non-stationary streams.

F5 — Vision Transformer backbone. The original paper provides preliminary results with ViT-B/16, showing that GMM-BA generalises beyond ResNet-50. Testing all six GMM-BA++ contributions with a ViT backbone is important, as transformer features occupy a different geometric space and may interact differently with L2 normalisation and covariance regularisation.

10. Conclusion

We presented **GMM-BA++**, a systematically improved version of the state-of-the-art online SF-UniDA method GMM-BA. Starting from a faithful reproduction of the original method (within $\leq 0.6\%$ on VisDA-C), we identified six concrete limitations through careful examination of the paper and codebase, and proposed a targeted fix for each.

Our contributions are summarised as follows. L2 feature normalisation and Tikhonov covariance regularisation (C1–C2) stabilise the GMM’s geometry and eliminate numerical failures. Confidence-weighted contrastive loss (C3) reduces the damage from noisy pseudo-labels by down-weighting uncertain samples. Adaptive per-class decay (C4) gives each class the right amount of memory based on how often it has been seen. Continual EMA threshold recalibration (C5) allows the OOD boundaries to track the improving GMM, yielding +9.4% PDA accuracy on VisDA-C. GPU-native batched likelihood (C6) achieves $2.5\times$ training speedup with no accuracy change.

Contributions C1–C4 are universally safe: no scenario degrades in our experiments. Contribution C5 is scenario-dependent: it benefits PDA strongly but requires tuning of α_{EMA} for ODA/OPDA scenarios, pointing to a scenario-adaptive threshold policy as the most promising direction for future work.

Overall, GMM-BA++ demonstrates that targeted, theory-motivated modifications to an already strong baseline can yield substantial improvements without sacrificing the memory efficiency or source-free property that make GMM-BA practically attractive.

Acknowledgements

We thank Pascal Schlachter for releasing the official GMM-BA codebase and pre-trained source models. This work was carried out as part of the Visual Computing course at IIT Hyderabad under the guidance of Prof. C. Krishna Mohan.

References

- [1] P. Schlachter, S. Wagner, and B. Yang. Memory-efficient pseudo-labeling for online source-free universal domain adaptation using a Gaussian mixture model. *arXiv:2407.14208*, 2024.

- [2] P. Schlachter and B. Yang. COMET: Contrastive mean teacher for online source-free universal domain adaptation. In *IJCNN*, 2024.
- [3] S. Qu, T. Zou, F. Roehrbein, C. Lu, G. Chen, D. Tao, and C. Jiang. Upcycling models under domain and category shift. *arXiv:2310.00946*, 2023.
- [4] S. Qu, T. Zou, F. Rohrbein, C. Lu, G. Chen, D. Tao, and C. Jiang. GLC++: Source-free universal domain adaptation through global-local clustering and contrastive affinity learning. *arXiv*, 2024.
- [5] S. Qu, T. Zou, L. He, F. Rohrbein, A. Knoll, G. Chen, and C. Jiang. LEAD: Learning decomposition for source-free universal domain adaptation. *arXiv*, 2024.
- [6] Y. Li, X. Xu, Y. Su, and K. Jia. On the robustness of open-world test-time training: Self-training with dynamic prototype expansion. In *ICCV*, pages 11836–11846, 2023.
- [7] J. Liang, D. Hu, and J. Feng. Do we really need to access the source data? source hypothesis transfer for unsupervised domain adaptation. In *ICML*, 2020.
- [8] D. Chen, D. Wang, T. Darrell, and S. Ebrahimi. Contrastive test-time adaptation. In *CVPR*, pages 295–305, 2022.
- [9] K. You, M. Long, Z. Cao, J. Wang, and M. I. Jordan. Universal domain adaptation. In *CVPR*, pages 2720–2729, 2019.
- [10] K. Saito and K. Saenko. OVANet: One-vs-all network for universal domain adaptation. In *ICCV*, pages 9000–9009, 2021.
- [11] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell. TENT: Fully test-time adaptation by entropy minimization. In *ICLR*, 2021.
- [12] X. Peng, B. Usman, N. Kaushik, D. Wang, J. Hoffman, and K. Saenko. VisDA: A synthetic-to-real benchmark for visual domain adaptation. In *CVPR Workshops*, 2018.
- [13] X. Peng, Q. Bai, X. Xia, Z. Huang, K. Saenko, and B. Wang. Moment matching for multi-source domain adaptation. In *ICCV*, pages 1406–1415, 2019.
- [14] Y. Ganin *et al.* Domain-adversarial training of neural networks. *JMLR*, 17(1):2096–2030, 2016.
- [15] M. Wang and W. Deng. Deep visual domain adaptation: A survey. *Neurocomputing*, 312:135–153, 2018.
- [16] K. Sohn *et al.* FixMatch: Simplifying semi-supervised learning with consistency and confidence. In *NeurIPS*, 2020.
- [17] A. N. Tikhonov and V. Y. Arsenin. *Solutions of Ill-Posed Problems*. Winston, 1977.
- [18] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, pages 770–778, 2016.
- [19] P. Khosla *et al.* Supervised contrastive learning. In *NeurIPS*, vol. 33, pages 18661–18673, 2020.